

SAADS db/ 模块详细设计方案

1. 设计目标

本方案面向 **WP1-1** 情报采集智能体的 Python 实现，目标不是简单“连库 + 执行 SQL”，而是把 PostgreSQL 中已经定义好的 wp11 schema，稳定映射为一套可维护、可测试、可被多个 Agent 复用的数据库访问层。

本设计直接对齐当前数据库对象：

- 来源采集层：source_type、intel_source、collection_task、raw_intel_record
- 攻击知识层：attack_entry、attack_cvss_assessment、attack_evidence、attack_taxonomy_map、attack_seed_asset、remediation_advice
- AI BOM 层：ai_component、ai_component_alias、attack_component_impact
- 治理审计层：dedup_audit、bom_resolution_queue
- 读模型/视图：v_primary_cvss_score、v_wp12_attack_feed、v_component_risk_overview、v_unresolved_bom_queue、v_source_quality_dashboard、mv_owasp_coverage

数据库模块需要解决的不是“怎么把所有 SQL 都塞进一个 repository.py”，而是：

1. 对采集 Agent、解析 Agent、BOM 解析 Agent、WP1-2 消费侧暴露清晰接口。
2. 把事务边界、连接池、异常、模型映射从业务逻辑中抽离出来。
3. 让 agents/intel_agents/ 中的多个子智能体共用一套数据库访问规范。
4. 保留 PostgreSQL 特性（JSONB、GIN、视图、物化视图、pg_trgm、gen_random_uuid()）的可用性，而不是被 ORM 过度抽象掉。

2. 设计原则

2.1 采用 psycopg3 + dataclass/Pydantic + Repository + Unit of Work

不建议把当前项目设计成“ORM-first”。原因如下：

- 当前 schema 已经相当明确，且使用了 PostgreSQL 特性；
- 表、视图、物化视图较多，读写模式并不均匀；
- 很多对象天然更适合 SQL 驱动，例如：
 - 基于视图的 WP1-2 投喂读取；
 - 去重审计与队列更新；
 - BOM 模糊匹配；
 - dashboard / overview 聚合读取。

因此建议：

- 底层驱动：psycopg (psycopg3)
- 连接池：psycopg_pool.ConnectionPool
- 领域模型：dataclass 和 pydantic (pydantic 仅用于边界输入校验，内部数据都是用 dataclass)
- 数据访问层：分域 Repository
- 事务管理：Unit of Work
- 服务层：跨表写入编排，不直接暴露给 SQL 层

2.2 Repository 按“领域子域”拆分，不按 CRUD 生硬拆分

不要使用单一的 repository.py 承载所有数据库操作。当前 schema 至少应拆为：

- 采集域 Repository
- 攻击知识域 Repository
- AI BOM 域 Repository
- 治理审计域 Repository
- 读模型 / 视图 Repository

2.3 读写分离

当前数据库对象天然存在两类访问路径：

- 写路径：面向表，强调事务一致性
- 读路径：面向视图 / 聚合视图，强调查询便利性

例如：

- attack_entry / attack_cvss_assessment / attack_evidence 是写路径
- v_wp12_attack_feed / v_component_risk_overview 是读路径

2.4 所有跨表写入必须经服务层或 UoW 完成

例如一次“原始情报解析入库”通常同时涉及：

- 写 raw_intel_record
- 写 attack_entry
- 写 attack_evidence
- 写 attack_cvss_assessment
- 写 attack_taxonomy_map
- 写 attack_component_impact

- 必要时写 `bom_resolution_queue`

这种逻辑不能散落在 Agent 代码里，必须集中在 `service + unit_of_work` 中。

3. 推荐目录结构

```
saads/
|
|─ agents/
|   └─ intel_agents/ （先不考虑这一部分）
|       │─ collector_agent.py
|       │─ parser_agent.py
|       │─ cvss_agent.py
|       │─ taxonomy_agent.py
|       │─ bom_mapper_agent.py
|       └─ seed_builder_agent.py
|
|─ db/
|   │─ __init__.py
|   │─ connection.py
|   │─ session.py
|   │─ exceptions.py
|   │─ unit_of_work.py
|   │─ pagination.py
|   │─ typing.py
|   │─ sql/
|   │   │─ attack_queries.py
|   │   │─ bom_queries.py
|   │   │─ source_queries.py
|   │   └─ governance_queries.py
|   │─ models/
|   │   │─ __init__.py
|   │   │─ source.py
|   │   │─ attack.py
|   │   │─ component.py
|   │   │─ governance.py
|   │   └─ views.py
|   │─ repositories/
```

```

|   |   |─ __init__.py
|   |   |─ base.py
|   |   |─ source_repository.py
|   |   |─ attack_repository.py
|   |   |─ component_repository.py
|   |   |─ governance_repository.py
|   |   |─ read_model_repository.py
|   |   └─ services/
|       |   |─ __init__.py
|       |   |─ ingestion_service.py
|       |   |─ attack_merge_service.py
|       |   |─ cvss_service.py
|       |   |─ taxonomy_service.py
|       |   |─ bom_resolution_service.py
|       |   └─ wp12_feed_service.py
|
└─ config.py

```

这个结构里，最关键的是：

- `models/`：定义 Python 侧数据对象
- `repositories/`：封装单域数据库访问
- `services/`：封装跨表业务事务
- `unit_of_work.py`：统一事务边界
- `read_model_repository.py`：专门读视图和物化视图

4. `config.py` 与数据库配置

数据库模块不应自己硬编码连接参数。建议把配置统一放在项目根部 `config.py`。

建议配置项：

```

POSTGRES_HOST
POSTGRES_PORT
POSTGRES_DB
POSTGRES_USER
POSTGRES_PASSWORD
POSTGRES_SCHEMA = "wp11"
POSTGRES_MIN_SIZE = 1
POSTGRES_MAX_SIZE = 10

```

```
POSTGRES_CONNECT_TIMEOUT = 5
POSTGRES_STATEMENT_TIMEOUT_MS = 30000
```

也可以补充：

```
POSTGRES_DSN
```

推荐优先使用 DSN。

5. db/connection.py: 连接池与连接工厂

职责

- 创建和持有 PostgreSQL 连接池
- 为上层提供连接获取接口
- 在连接建立后设置：
 - search_path = wp11, public
 - statement timeout
 - row factory (dict row 或 class row)

设计要点

1. 使用全局单例连接池，但初始化动作应显式完成。
2. 避免在 import 时自动建连。
3. 提供同步上下文管理接口。

推荐接口

- init_pool()
- close_pool()
- get_connection()
- connection_context()

不要做的事

- 不要在每个 repository 里自己 `psycopg.connect(...)`
 - 不要在 Agent 里直接管理连接生命周期
-

6. db/session.py: 游标与执行辅助

职责

- 统一 cursor 的创建方式
- 统一 SQL 执行的参数绑定与日志记录
- 提供常见执行封装:
 - fetch_one
 - fetch_all
 - execute
 - execute_many
 - fetch_scalar

为什么需要这一层

因为当前系统里会有大量“读一个视图”“按条件更新一个队列状态”“执行 upsert”的小型操作。如果所有 repository 都自己处理 cursor，会重复很多错误处理和日志代码。

7. db/exceptions.py: 异常体系

建议定义数据库模块自己的异常层级，避免上层 Agent 直接依赖 psycopg 原始异常。

推荐异常类型

- DatabaseError: 数据库模块基类异常
- ConnectionInitError
- QueryExecutionError
- NotFoundError
- DuplicateEntityError
- ValidationError
- ConcurrencyConflictError
- TransactionError

设计原则

- psycopg 的异常在 repository 内部转换为项目级异常
- Agent 只处理项目级异常，不感知底层驱动细节

8. db/unit_of_work.py: 事务边界核心

作用

统一管理跨 Repository 的事务。

为什么必须有 UoW

下面这些动作天然就是“一次事务”：

1. 新建 attack_entry
2. 写入证据 attack_evidence
3. 写入主 CVSS attack_cvss_assessment
4. 写入 taxonomy attack_taxonomy_map
5. 写入组件影响 attack_component_impact
6. 必要时创建 bom_resolution_queue

如果中间任何一步失败，前面的写入必须回滚。

推荐能力

- `__enter__` / `__exit__`
- 自动 commit / rollback
- 暴露按域组织的 repositories，例如：
 - `uow.sources`
 - `uow.attacks`
 - `uow.components`
 - `uow.governance`
 - `uow.read_models`

一个 UoW 中不应做的事

- 不要在长时间网络请求外持有事务
 - 采集 HTTP 拉取、LLM 推理、规则计算等慢操作，应在事务外完成；事务内只做确定的数据库读写
-

9. db/models/: Python 数据模型设计

这一层负责从 PostgreSQL 的物理表映射到 Python 可操作对象。

推荐使用 `dataclass` 表达“数据库记录对象”。如果边界输入复杂，可额外定义 `pydantic` 输入 DTO。

10. db/models/source.py

建议模型

- `SourceType`
- `IntelSource`
- `CollectionTask`
- `RawIntelRecord`

主要用途

- 采集源注册
- 采集任务生命周期管理
- 原始情报入库

特别说明

`RawIntelRecord` 建议包含:

- `raw_id`
- `source_id`
- `task_id`
- `source_uri`
- `title`
- `content_hash`
- `raw_format`
- `payload_uri`
- `language_code`
- `relevance_score`
- `parser_status`
- `fetch_at`
- `created_at`

- is_deleted
-

11. db/models/attack.py

建议模型

- AttackEntry
- AttackCvssAssessment
- AttackEvidence
- AttackTaxonomyMap
- AttackSeedAsset
- RemediationAdvice

设计说明

AttackEntry 对应标准化攻击知识主表，是整个 WP1-1 的核心模型。

AttackCvssAssessment 必须独立为单独模型，不要并入 AttackEntry。原因：

- 一条攻击条目可以有多条 CVSS 记录
- 同一攻击可能同时存在 supplied / calculated / estimated / manual
- is_primary 只标识对外主分数，不等于唯一分数

AttackEvidence 这是攻击条目与原始情报之间的证据链表，不能省略，否则 attack_entry 会失去可追溯性。

AttackSeedAsset 这是 WP1-2 的关键供给对象，代表 PoC、payload 模板、prompt 语料、rule 等“测试种子”。

12. db/models/component.py

建议模型

- AiComponent
- AiComponentAlias

- `AttackComponentImpact`

设计说明

这一组模型服务 AI BOM 适配逻辑。

AiComponent 标准 AI 组件主表。

AiComponentAlias 负责别名规范化与模糊匹配。由于 schema 上已经有 `normalized_alias + tri-gram` 索引，这意味着 Python 侧必须明确区分：

- 原始别名
- 规范化别名

AttackComponentImpact 用于记录“某攻击影响哪些组件以及以何种版本约束影响”。这是 AI BOM 风险联动的关键桥表。

13. db/models/governance.py

建议模型

- `DedupAudit`
- `BomResolutionQueueItem`

作用

治理审计模型不是附属信息，而是运营层对象：

- **DedupAudit**：记录去重决策
- **BomResolutionQueueItem**：记录 AI BOM 未解析项的人工/半自动复核队列

这些对象通常会被独立的治理 Agent 或后台管理界面消费。

14. db/models/views.py

建议把视图映射对象集中在这里：

- PrimaryCvssView
- Wp12AttackFeedRow
- ComponentRiskOverviewRow
- UnresolvedBomQueueRow
- SourceQualityDashboardRow
- OwaspCoverageRow

设计原则

- 视图模型是只读对象
 - 不参与写回
 - 单独放置，避免与表模型混淆
-

15. db/repositories/base.py

作用

封装所有 repository 的公共能力：

- 持有 connection / session
- 执行 SQL
- 统一日志与异常转换
- 常见辅助函数（唯一查找、分页、批量插入）

建议公共方法

- _fetch_one(...)
 - _fetch_all(...)
 - _execute(...)
 - _execute_many(...)
 - _fetch_scalar(...)
-

16. source_repository.py

负责对象

- source_type
- intel_source
- collection_task
- raw_intel_record

建议接口

- get_source_by_name(source_name)
- list_enabled_sources(source_type=None)
- create_collection_task(...)
- update_collection_task_status(...)
- insert_raw_intel_record(...)
- get_raw_record_by_hash(source_id, content_hash)
- mark_raw_record_parser_status(raw_id, status)
- list_pending_raw_records(limit)

业务特点

这一层是采集智能体与数据库的第一接触面。重点是：

- 控制重复原始数据写入
- 管理 task 生命周期
- 为解析 Agent 提供待处理原始记录

17. attack_repository.py

负责对象

- attack_entry
- attack_cvss_assessment
- attack_evidence
- attack_taxonomy_map
- attack_seed_asset
- remediation_advice

建议接口

- `get_attack_by_code(attack_code)`
- `create_attack_entry(...)`
- `update_attack_entry(...)`
- `upsert_attack_entry_by_code(...)`
- `insert_attack_evidence(...)`
- `list_attack_evidence(attack_id)`
- `insert_cvss_assessment(...)`
- `list_cvss_assessments(attack_id)`
- `set_primary_cvss(score_id)`
- `replace_primary_taxonomy(...)`
- `insert_seed_asset(...)`
- `list_published_seed_assets(attack_id)`
- `insert_remediation_advice(...)`

设计细节

关于 **`set_primary_cvss(score_id)`** 由于 schema 上存在 `uq_cvss_primary_per_attack_version`, 因此切换主 CVSS 时必须:

1. 先找到目标记录的 `attack_id` 与 `cvss_version`
2. 把同攻击、同版本的其他记录置为 `is_primary = false`
3. 再把目标记录置为 `true`

必须在同一事务中执行。

关于 **`taxonomy`** `attack_taxonomy_map` 同样有“每种 `taxonomy_type` 只能一个主项”的约束, 因此 `replace_primary_taxonomy(...)` 也应走事务。

18. component_repository.py

负责对象

- `ai_component`
- `ai_component_alias`
- `attack_component_impact`

建议接口

- `get_component_by_code(component_code)`
- `get_component_by_name(component_name)`
- `search_component_alias(normalized_alias, limit=10)`
- `create_component(...)`
- `insert_component_alias(...)`
- `list_component_aliases(component_id)`
- `upsert_attack_component_impact(...)`
- `list_component_impacts_by_attack(attack_id)`
- `list_attacks_by_component(component_id)`

设计细节

由于 `ai_component_alias.normalized_alias` 是全局唯一，别名标准化函数必须稳定，否则会导致重复别名和解析错误。

建议在 Python 层统一实现：

- 小写化
 - 去空格 / 下划线 / 连字符标准化
 - 去 vendor 前后冗余前缀
-

19. governance_repository.py

负责对象

- `dedup_audit`
- `bom_resolution_queue`

建议接口

- `insert_dedup_audit(...)`
- `list_dedup_candidates_for_review(...)`
- `enqueue_bom_resolution(...)`
- `list_open_bom_queue(limit)`
- `resolve_bom_queue_item(queue_id, resolved_component_id)`
- `reject_bom_queue_item(queue_id)`

设计说明

这里不应只做“后台管理接口”。对于 `bom_mapper_agent` 来说，无法自动解析的组件必须立即写入 `queue`，而不是静默失败。

20. read_model_repository.py

负责对象

- `v_primary_cvss_score`
- `v_wpl2_attack_feed`
- `v_component_risk_overview`
- `v_unresolved_bom_queue`
- `v_source_quality_dashboard`
- `mv_owasp_coverage`

建议接口

- `get_primary_cvss(attack_id)`
- `list_wpl2_attack_feed(filters...)`
- `list_component_risk_overview(...)`
- `list_unresolved_bom_queue(...)`
- `get_source_quality_dashboard(...)`
- `list_owasp_coverage(...)`
- `refresh_mv_owasp_coverage()`

为什么单独拆分

因为这些查询面向消费方：

- WP1-2
- dashboard
- 风险总览
- 运营分析

它们的目标不是保持表写入一致性，而是提供稳定的读模型。

21. db/services/: 服务层设计

服务层负责“跨表业务动作”的编排。建议至少有以下服务：

21.1 ingestion_service.py

负责：

- 创建采集任务
- 原始记录入库
- 防重复检查
- parser 状态更新

21.2 attack_merge_service.py

负责：

- 将解析结果合并成 attack_entry
- 写证据链
- 建立去重审计记录

21.3 cvss_service.py

负责：

- 写入 CVSS
- 计算/补全分数
- 切换主评分

21.4 taxonomy_service.py

负责：

- 批量写 taxonomy
- 替换主 taxonomy

21.5 bom_resolution_service.py

负责：

- 组件别名匹配
- 自动解析成功则写 `attack_component_impact`
- 自动解析失败则写 `bom_resolution_queue`

21.6 wp12_feed_service.py

负责:

- 面向 WP1-2 构造“测试脚本生成输入集”
 - 优先通过 `v_wp12_attack_feed` 提供读取接口
-

22. SQL 组织方式: db/sql/

不建议把所有 SQL 内联进 repository 方法体。建议把复杂 SQL 放入 `db/sql/*.py` 中按领域维护。

优点

- SQL 可测试
- SQL 与控制流解耦
- 复杂查询（尤其视图查询、模糊匹配、聚合）更易维护

适合抽离的 SQL

- feed 查询
 - alias 模糊匹配
 - dedup 候选查询
 - dashboard 聚合读取
-

23. 事务边界建议

23.1 一条原始情报抓取任务

应拆成两个阶段:

阶段 A: 事务外

- 发 HTTP 请求
- 下载 payload
- 算 hash
- 解析内容
- 调用模型 / LLM

阶段 B: 事务内

- 插入 collection_task 更新状态
- 插入 raw_intel_record
- 插入 / 更新 attack_entry
- 插入 attack_evidence
- 插入 attack_cvss_assessment
- 插入 taxonomy / component impact
- 更新 task 状态

23.2 不建议在一个事务中同时做

- 网络访问
- 长时间文件 IO
- LLM 推理
- 数据库写入

否则锁持有时间过长。

24. 并发与幂等性设计**24.1 原始记录幂等**

基于:

- raw_intel_record (source_id, content_hash) 唯一约束

因此 Repository 应优先提供:

- insert_or_get_raw_record(...)

24.2 别名幂等

基于:

- `ai_component_alias.normalized_alias` 唯一约束

因此 `insert_component_alias(...)` 应处理重复别名异常。

24.3 攻击-组件影响幂等

基于:

- `uq_attack_component_impact_dedup`

因此建议使用 `upsert` 逻辑而非裸插入。

25. 日志与可观测性

数据库模块至少应记录:

- SQL 模块名 / repository 名
- `trace_id` (若存在)
- `task_id` / `attack_id` / `raw_id` / `component_id`
- 执行耗时
- 行数影响
- 异常类型

注意

不要把完整 `payload`、敏感 `token`、长文本摘要直接打进日志。

26. 测试策略

单元测试

- 测试模型映射
- 测试 `alias` 规范化
- 测试 SQL 过滤条件

- 测试异常转换

集成测试

- 使用测试库执行真实 SQL
- 校验唯一约束、外键、事务回滚行为
- 校验视图查询

建议重点测试场景

1. 原始记录重复写入
 2. 主 CVSS 切换
 3. 主 taxonomy 替换
 4. 无法解析的 BOM 入 queue
 5. v_wp12_attack_feed 是否只返回可供 WP1-2 使用的资产
-

27. 与 agents/intel_agents/ 的边界

Agent 负责

- 调外部源
- 调规则 / LLM
- 生成标准化结果对象
- 决定何时入库

db/ 负责

- 连接
- 事务
- SQL
- 模型映射
- 读写一致性
- 视图读取

一个非常重要的原则

Agent 不应直接拼表名写 **SQL**。

Agent 只能：

- 调 service
 - 或调 repository（只在非常简单的读场景）
-